

Todo App

İyileştirme Önerileri

15 öneri • Önceliklendirilmiş • Kavramsal açıklamalı

KRİTİK

Bug veya HTML/UX hatası. Mutlaka düzelt.

ÖNEMLİ

Code quality, mimari. Daha temiz/sağlam kod için.

POLISH

İyileştirme, polish. Yapmasan da çalışır, ama iyi olur.

Önerileri sırayla uygula. Her birinde önce kavramı anla, sonra kendi yorumunla yaz. Çözüm kodu vermedim — kendi mühendisliğini geliştirmek için.

İçindekiler

#	Konu	Öncelik
1	ToDoList'te Aynı ID Sorunu	KRİTİK
2	Form Submit preventDefault Eksik	KRİTİK
3	Submit Sonrası Form Reset Yok	KRİTİK
4	Routing Artık Gereksiz	ÖNEMLİ
5	update() Metodu Hala Sıra Bozuyor	ÖNEMLİ
6	TodoFilter İki Sorumluluk Birden	ÖNEMLİ
7	ReactiveFormsModule Gereksiz Import	POLISH
8	crypto.randomUUID() Gereksiz Method Chain	POLISH
9	onsubmit Naming Convention	POLISH
10	TodoItem Ayır Component Olmalı	POLISH
11	Validation İyileştir (computed + disabled)	POLISH
12	localStorage Type Guard Ekle	POLISH
13	Date Object Parse'tan Sonra Çevir	POLISH
14	ToggleTodoInput Yerine id: string Yeterli	POLISH
15	Validation İçinde Side-Effect (Title Set)	POLISH

KRİTİK

#1 — TodoList'te Aynı ID Sorunu

❏ Sorun

TodoList template'inde her todo için aynı `id="todo-1"` ve `for="todo-1"` kullanılıyor. Bu HTML hatası — bir sayfada aynı ID birden fazla element'te olamaz.

```
<!-- ❏ Şu an -->
<input type="checkbox" id="todo-1" ...>
<label for="todo-1" ...>{{ todo.title }}</label>

<!-- 5 todo varsa, 5 element aynı id'ye sahip! -->
```

⚠ Etkileri

- HTML validator hata verir
- Accessibility tools (ekran okuyucu) bozulur
- Label'a tıklayınca ilk checkbox seçilir (kullanıcının istediği değil)
- Bazı tarayıcı davranışları beklenmedik olur

❏ Çözüm İpucu

Her todo'nun zaten kendi unique id'si var: `todo.id`. Onu kullan. HTML attribute'larına dinamik değer atamak için Angular'da hangi syntax'ı kullanırsınız?

```
<!-- ✓ Doğru pattern -->
<input type="checkbox" [id]="???" ...>
<label [for]="???" ...>{{ todo.title }}</label>
```

❏ Düşünme Sorusu

Sadece `[id]="todo.id"` yazsan yeterli mi? Yoksa prefix eklemek mantıklı mı (örn. `[id]=" 'todo-' + todo.id"`)? Sayfada başka yerde de UUID kullanıyor olabilirsin — prefix çakışmayı engeller.

KRİTİK

#2 — Form Submit'te preventDefault Eksik

❏ Sorun

`<form (submit)="onsubmit()">` kullanıyorsun ama `event.preventDefault()` çağırılmıyorsun. Form'un default davranışı submit edildiğinde sayfayı reload etmektir.

⚠ Etkileri

- Ekle butonuna basınca sayfa yeniden yüklenir
- In-memory state (eğer localStorage'a kaydedilmediyse) kaybolur
- URL'de query parametre olarak değerler görünebilir

- Kullanıcı deneyimi bozulur — yavaş, kötü

❑ Test Et

F12 ile DevTools aç → Network sekmesini izle → Form submit et. Yeni request görüyorsan reload oluyor demektir.

❑ Çözüm İpucu

Method'a Event parametresi al ve `preventDefault()` çağır:

```
// Template
<form (submit)="onSubmit($event)">

// Component
onSubmit(event: Event): void {
  event.preventDefault(); // ← Sayfa reload'unu engeller
  ...
}
```

KRİTİK

#3 — Submit Sonrası Form Reset Yok

❑ Sorun

Kullanıcı yeni todo ekledi → input'ta hala önceki yazı duruyor → priority hala "high"da. Yeni todo eklemek için kullanıcı önce form'u manuel temizlemek zorunda kalıyor.

❑ Çözüm İpucu

Emit'ten sonra signal'ları reset et:

```
onSubmit(event: Event): void {
  event.preventDefault();
  if (this.title().trim().length === 0) return;

  this.addToDo.emit({
    title: this.title().trim(),
    completed: false,
    priority: this.selectedPriority(),
  });

  // ✓ Form'u reset et
  this.title.set('');
  this.selectedPriority.set(???); // Default priority
}
```

❑ Tasarım Kararı

Priority'yi reset etmeli misin? İki yaklaşım var:

- Reset et: Form temizlensin, her seferinde aynı şekilde başlasın
- Reset etme: Kullanıcı arka arkaya aynı priority'li todo ekliyorsa kolaylık sağla

İkisi de geçerli — UX kararın.

ÖNEMLİ

#4 — Routing Artık Gereksiz

❏ Sorun

Home component'i kaldırdın, tek sayfa uygulaması artık. Ama `app.config.ts`'te hala `provideRouter(routes)` var, `app.routes.ts` dosyası var (boş array ile). Bu gereksiz kod.

⚠ Etkileri

- Bundle size'da gereksiz router kodu
- Yeni okuyucu için kafa karıştırıcı: "Routing var ama route yok, neden?"
- Dead code maintenance yükü

❏ Yapılacaklar

- `app.routes.ts` dosyasını sil
- `app.config.ts`'te `provideRouter(routes)` satırını sil
- `import { provideRouter } satırını sil`
- `import { routes } satırını sil`

❏ İleride

Eğer ileride birden fazla sayfa eklersen (örn: `/settings`, `/about`), routing'i geri ekleyebilirsin. `ng generate` komutları bu konuda yardımcı olur.

ÖNEMLİ

#5 — `update()` Metodu Hala Sıra Bozuyor

❏ Sorun

`TodoService`'in `update()` metodu `filter` + `spread` kullanıyor. Bu, güncellenen `todo`'yu listenin sonuna atar — sıra bozulur.

```
// ❌ Şu anki kod
this._todos.update((todos) => [
  ...todos.filter((t) => t.id !== todoInput.id), // ← Eskiye sil
  { ...currentTodo, ...todoInput }, // ← Yeniyi sona ekle
]);

// Liste: A, B, C, D, E → B güncellenirse → A, C, D, E, B (yanlış!)
```

❌ İlginç Tutarsızlık

`toggle()` metodunda `map` kullanmışsın — doğru! Ama aynı pattern'i `update()`'e uygulamamışsın. Tutarlılık için ikisini de aynı şekilde yaz.

❌ Çözüm İpucu

`map` kullan — her item için "değiştir veya aynı bırak" kararı verir, sıra korunur:

```
// ✓ Doğru pattern (toggle ile aynı)
update(todoInput: UpdateTodoInput): void {
  this._todos.update((todos) =>
    todos.map((t) =>
      t.id === todoInput.id ? { ...t, ...todoInput } : t
    )
  );
  this.saveTodos();
}
```

ÖNEMLİ

#6 — TodoFilter İki Sorumluluk Birden

❌ Sorun

`TodoFilter` component'inin template'inde iki ayrı section var: filter butonları + istatistikler. Bu Single Responsibility Principle (SRP) ihlali.

```
<!-- TodoFilter component'i içinde -->
<section class="todo-filter">
  <!-- Filter butonları -->
</section>

<section class="todo-stats">
  <!-- İstatistikler -->
</section>
```

⚠ Neden Sorun?

- Component iki farklı amaçta hizmet ediyor (filter + display)
- İsmi yanıltıcı: "TodoFilter" ama stats de yapıyor
- 6 input alıyor (total, active, completed, high, medium, low) — çok fazla
- Test etmek zor: hem filter hem stat logic'i birlikte

- Reuse zor: stats'i başka yere koymak istersen mecbur kalıyorsun ayırırsın

❏ Çözüm — İki Component'e Böl

- TodoFilter — sadece filter butonları (total, active, completed input + filterChange output)
- TodoStats — sadece istatistikler (total, active, completed, high, medium, low input)

❏ Bonus İpucu

6 ayrı input yerine, service'te tek bir stats computed yapabilirsin. Object olarak döndür: { total, active, completed, byPriority: { low, medium, high } }. Tek input olur, daha temiz.

POLISH

#7 — ReactiveFormsModule Gereksiz Import

❏ Sorun

TodoFormComponent'te ReactiveFormsModule import ediliyor, ama template'te kullanılmıyor ([formGroup], ngModel, formControl yok). Manuel binding kullanıyorsun.

```
// ❏ Şu anki kod
import { ReactiveFormsModule } from '@angular/forms';

@Component({
  imports: [ReactiveFormsModule], // ← Gereksiz
  ...
})
```

⚠ Etkileri

- Bundle size'da gereksiz kod
- Production build'de tree-shaking yardım eder ama dev mode'da yavaşlatır
- Yeni okuyucu: "Reactive Forms mı kullanıyor?" diye yanılır

❏ Çözüm

Hem import'u hem imports array'inden kaldır:

```
// ✓ Sade hali
@Component({
  imports: [], // Boş veya hiç yazma
  ...
})
```

❏ Sorun

```
id: crypto.randomUUID().toLowerCase().toString(),  
  
// Neden .toLowerCase()?  
// Neden .toString()?
```

❏ Gerçek

- crypto.randomUUID() zaten lowercase string döner
- "3f8b2c4e-1234-5678-9abc-def012345678" formatında
- .toLowerCase() → zaten lowercase, gereksiz
- .toString() → zaten string, gereksiz

❏ Çözüm

```
// ✓ Sade hali  
id: crypto.randomUUID(),
```

❏ Genel Prensiptir

Bilmediğin bir fonksiyonu kullanırken, önce return type'ını öğren. MDN veya TypeScript'in autocomplete'i sana söyler. Gereksiz chain'ler "defensive coding"ın yanlış uygulanması.

❏ Sorun

onsubmit tek kelime — DOM property ismi. JavaScript'te element.onsubmit = function() {} şeklinde kullanılıyor (klasik event handler). Senin method'unun ****camelCase**** olması gerekir.

```
// ❏ Confusing  
onsubmit(): void { ... }  
  
// ✓ Standart  
onSubmit(): void { ... }
```

❏ Neden Önemli?

- TypeScript convention: method/variable isimleri camelCase
- onsubmit DOM property'si — çakışma riski

- Okuyan dev "DOM property mi yoksa custom method mi?" diye düşünür
- Genel pattern: onClick, onChange, onSubmit, onDelete...

❏ Yapılacak

IDE'nin "Rename Symbol" özelliği (F2 veya right-click → Rename) ile method'u tek seferde tüm dosyalardaki referanslarıyla birlikte yeniden adlandır.

POLISH

#10 — TodoItem Ayır Component Olmalı

❏ Sorun

Şu an TodoList component'in template'inde @for ile her todo'yu inline render ediyorsun. Mantığın todo'ya ait UI ve liste mantığı iç içe.

```
<!-- ❏ Şu an: Her şey TodoList içinde -->
@for (todo of todos(); track todo.id) {
  <div class="todo-item" ...>
    <input type="checkbox" ...>
    <label>{{ todo.title }}</label>
    <span class="priority-pill" ...>{{ todo.priority }}</span>
    <button>x</button>
  </div>
}
```

⚠ Neden Sorun?

- Inline edit (update) eklediğinde template kocaman olur
- Her todo'nun kendi state'i (örn. isEditing) lazımsa nereye koyacaksın?
- TodoItem mantığı TodoList'le karışıyor
- Test etmek zor: tek bir todo davranışını izole edemezsin
- Reuse zor: aynı UI başka yerde lazım olursa kopyalamak gerekir

❏ Çözüm — TodoItem Component'i Yarat

CLI ile yarat:

```
ng g c components/todo-item
```

TodoItem'da olacaklar:

- todo = input.required<Todo>()
- toggled = output<ToggleTodoInput>()
- deleted = output<string>()
- Edit özelliği eklenince: updated = output<UpdateTodoInput>()
- priorityLabel computed signal (Türkçe label için)
- Edit özelliği eklenince: isEditing state

□ Inline Edit İçin Gerekli

Sen "tıklayınca edit edilebilir olsun" istiyordun. Bu özellik için TodoItem ayrı component olmalı. Çünkü her todo'nun kendi `isEditing` state'i olacak. TodoList içinde tek bir state ile yönetmek karmaşık olur.

POLISH

#11 — Validation İyileştir (computed + disabled)

□ Sorun

Form'un Ekle butonu her zaman aktif. Boş title ile bile basılabilir. Validation sadece `onInputChange`'de yapılıyor, o da problemli.

□ Modern Yaklaşım

Bir computed signal yarat — form geçerli mi diye söylesin. Button'a bağla.

```
// Component
isValid = computed(() => this.title().trim().length > 0);

// Template
<button type="submit" [disabled]="!isValid()">Ekle</button>
```

✂ Avantajları

- Title değişince otomatik yeniden hesaplanır (reactive)
- Button görsel olarak da disabled görünür → kullanıcı anlar
- Submit'te ekstra check'e gerek yok (button basılamaz)
- Single source of truth — validation logic tek yerde

□ Bonus

İleride başka validation'lar eklersen (örn. max length, special chars) `isValid` computed'ında birleştirirsin. Template hiç değişmez. Separation of concerns.

POLISH

#12 — localStorage Type Guard Ekle

□ Sorun

```
// ☐ Şu anki kod
private getTodos(): Todo[] {
  const storageValues = localStorage.getItem(...);
  return storageValues ? JSON.parse(storageValues) : [];
  // ~~~~~
  // any döner, Todo[] olduğunu
  // varsayıyorsun!
}
```

⚠ Riskler

- Kullanıcı DevTools'tan localStorage'ı bozarsa runtime error
- Eski sürüm veri yapısı uyumsuz olursa app çöker
- Test ederken yanlış data localStorage'a yazıldıysa debug zorlaşır
- TypeScript tipi yalan söylüyor — runtime'da Todo[] olmayabilir

☐ Çözüm — Type Guard (Modül 2.5)

```
// models/todo.model.ts
export function isTodo(obj: unknown): obj is Todo {
  if (typeof obj !== 'object' || obj === null) return false;
  const t = obj as Record<string, unknown>;
  return (
    typeof t['id'] === 'string' &&
    typeof t['title'] === 'string' &&
    typeof t['completed'] === 'boolean' &&
    /* priority kontrolü, vs */
  );
}

export function isTodoArray(arr: unknown): arr is Todo[] {
  return Array.isArray(arr) && arr.every(isTodo);
}
```

```
// service'te
private getTodos(): Todo[] {
  const raw = localStorage.getItem(...);
  if (!raw) return [];

  try {
    const parsed: unknown = JSON.parse(raw);
    if (!isTodoArray(parsed)) {
      console.warn('Invalid todos in storage');
      return [];
    }
    return parsed;
  } catch {
    return [];
  }
}
```

❏ Sorun

JSON'da Date object yok — string olarak saklanıyor. localStorage'dan okurken JSON.parse() geri okurken string olarak veriyor. Senin Todo.createdAt tipi Date ama runtime'da string.

```
// Veri saklarken:
JSON.stringify({
  createdAt: new Date(), // Date object
});
// → '{"createdAt":"2026-05-19T12:34:56.789Z"}' (string)

// Veri okurken:
const parsed = JSON.parse(raw);
// parsed.createdAt → "2026-05-19T..." ← STRING!
// .getTime() çağırırsan → TypeError!
```

❏ Çözüm

Parse'tan sonra Date object'e çevir:

```
return parsed.map((t: any) => ({
  ...t,
  createdAt: new Date(t.createdAt), // ← Date object'e çevir
}));
```

❏ Şimdi Önemli mi?

Şu an createdAt'i kullanmıyorsun, bu yüzden bug görünür değil. Ama ileride sıralama (yeni todayu üstte göster) veya "X gün önce" gösterimi eklersen date.getTime() hata verecek. Şimdi düzelt, sonra hatırlamak zor olur.

❏ Sorun

Toggle için ToggleTodoInput = Pick<Todo, 'id' | 'completed'> tipi yarattın. Ama bu gereksiz karmaşıklık. Service zaten current değeri biliyor — tersini hesaplayabilir.

```
// ❌ Şu anki yaklaşım
// TodoItem
onChange(event: Event, id: string): void {
  const target = event.target as HTMLInputElement;
  this.toggleTodo.emit({ id, completed: target.checked });
  // ~~~~~
  // Yeni değer hesaplanıyor
}

// Service
toggle(input: ToggleTodoInput): void {
  // input.completed kullanılıyor (component hesapladı)
}
```

⚠ Neden Suboptimal?

- İki yerde state takibi: hem checkbox'ın hem service'in
- Race condition riski: Eğer checkbox değeri ile service değeri uyuşmazsa?
- Component UI mantığı ile business mantığını karıştırıyor
- Daha az kod aslında daha güvenli

✅ Daha Temiz Yaklaşım

Sadece id gönder. Service current değeri okur, tersini set eder:

```
// ✓ TodoItem
onToggle(): void {
  this.toggled.emit(this.todo().id);
}

// ✓ Service
toggle(id: string): void {
  this._todos.update((todos) =>
    todos.map((t) =>
      t.id === id ? { ...t, completed: !t.completed } : t
    )
  );
}
```

✅ Bonus

ToggleTodoInput tipini de silebilirsin. Daha az kod, daha az tip, daha sade.

POLISH

#15 — Validation İçinde Side-Effect (Title Set)

❌ Sorun

```
// ❌ Mevcut kod
onInputChange(event: Event) {
  const target = event.target as HTMLInputElement;
  if (!target) return;
  if (target.value.length === 0 || target.value === ' ') return;
  this.title.set(target.value);
}
```

❌ Bunun Yarattığı Bug

Kullanıcı şöyle yapsa:

- "ABC" yazar → signal: "ABC"
- Tek tek siler: "AB" → signal: "AB" ✓
- Daha sil: "A" → signal: "A" ✓
- Son karakteri sil: "" → signal: hala "A"! (early return)
- Input görsel olarak boş, ama signal'da "A" var

Input görsel olarak boş ama signal'da eski değer kaldı. Synchronization sorunu.

❌ Sorunun Kaynağı

Validation'ı onInputChange'de yapıyorsun ama yanlış yere yapıyorsun. Validation "hangi değerlerin signal'a yazılacağına" karar etmemeli — her değişikliği signal'a yansıtsın, validation'ı submit'te yap.

❌ Doğru Pattern

```
// ✓ Doğru
onInputChange(event: Event) {
  const target = event.target as HTMLInputElement;
  this.title.set(target.value); // Her zaman set et
}

// Validation computed olarak (önceki #11 ile birleşik)
isValid = computed(() => this.title().trim().length > 0);

onSubmit(event: Event): void {
  event.preventDefault();
  if (!this.isValid()) return; // ← Validation burada
  this.addToDo.emit({
    title: this.title().trim(), // ← trim burada
    ...
  });
}
```

❌ Genel Prensiptir

"Her input değişimi signal'a yansısın, validation submit'te yapılsın" — bu modern form yaklaşımı. UI durumu (input) ve veri durumu (signal) her zaman senkron. Validation kararı olmalı, filtre değil.

□ Bonus: Inline Edit Özelliği

İstediğin özellik: Kullanıcı todo text'ine tıklayınca düzenlenebilir olsun, enter'a basınca yeni text geçerli olsun.

□ State Modeling

TodoItem component'te 2 signal gerekli:

```
isEditing = signal(false); // Edit mode aktif mi?  
editText = signal(''); // Edit ediliyor olan değer (buffer)
```

Neden buffer? Eğer kullanıcı yazıyor ve sonra vazgeçerse, ana todo.title'a dokunmamalısın. Buffer'da değişiklikleri tut, Enter'da kalıcı yap.

□ Template — @if ile İki Mod

```
@if (isEditing()) {  
  <!-- Edit mode -->  
  <input  
    type="text"  
    [value]="editText()"  
    (input)="onEditChange($event)"  
    (keydown.enter)="saveEdit()"  
    (keydown.escape)="cancelEdit()"  
    (blur)="cancelEdit()"  
    autofocus  
  />  
} @else {  
  <!-- Normal mode -->  
  <label class="todo-title" (click)="startEdit()">  
    {{ todo().title }}  
  </label>  
}
```

□ Method'lar — 4 Adet

1. startEdit()

Tıklamada çağrılır. Buffer'ı mevcut title ile doldur, edit mode'u aktif et.

```
startEdit(): void {  
  this.editText.set(this.todo().title); // Buffer = mevcut title  
  this.isEditing.set(true);  
}
```

2. onEditChange(event)

Input'taki her değişimi buffer'a yansıt.


```
onEditChange(event: Event): void {  
  const target = event.target as HTMLInputElement;  
  this.editText.set(target.value);  
}
```

3. saveEdit()

Enter'da çağrılır. Validate et, geçerliyse parent'a emit et veya service'e söyle.

```
saveEdit(): void {  
  const trimmed = this.editText().trim();  
  if (trimmed.length === 0) return; // Boşsa kaydetme  
  if (trimmed === this.todo().title) {  
    this.isEditing.set(false); // Değişmemişse sadece edit'ten çık  
    return;  
  }  
  
  // Parent'a haber ver  
  this.updated.emit({ id: this.todo().id, title: trimmed });  
  this.isEditing.set(false);  
}
```

4. cancelEdit()

Escape'te veya blur'da çağrılır. Sadece edit mode'u kapat.

```
cancelEdit(): void {  
  this.isEditing.set(false);  
  // editText'i reset etmeye gerek yok – sonraki startEdit'te  
  // yeniden mevcut title ile dolar  
}
```

□ Keyboard Event Binding

Angular'da klavye event'leri için directive-style syntax:

```
(keydown.enter)="saveEdit()" // Sadece Enter'da  
(keydown.escape)="cancelEdit()" // Sadece Escape'te  
(keydown.tab)="..." // Sadece Tab'de  
(keydown.control.s)="..." // Ctrl+S  
(keydown.shift.enter)="..." // Shift+Enter
```

Angular otomatik filter eder. JavaScript'te if (event.key === 'Enter') yazmana gerek yok.

□ Önemli Tasarım Kararları

Click veya Double-click?

Tek click kazara tetiklenebilir. Double-click daha güvenli ama keşfedilmesi zor. Tasarım gereği — tek click ile başla, kullanıcılar şikayet ederse double-click yap.

Blur'da Save mi Cancel mi?

Bu UX kararı. Bazı uygulamalar (Notion gibi) blur'da otomatik save eder. Bazıları (TodoMVC) cancel eder. İlk versiyonda cancel daha güvenli — kullanıcı kasıtsız tıklayıp dışarı çıkarsa değişiklikleri kaybetmez.

Boş Bırakırsa?

Boş title geçersiz. Tasarım kararı: ya save etme (kullanıcı vazgeçer) ya da silme onayı göster ("Todo silinsin mi?"). İlki daha basit.

Title Değişmemişse?

Eğer trimmed === todo.title ise, save'i atla, sadece edit mode'dan çık. Gereksiz update engellenir.

Autofocus?

Edit açılınca input'a focus verilmeli. HTML'in autofocus attribute'u genelde çalışır. Daha sağlam yöntem için ViewChild + ElementRef gerekli (ileri konu).

□ Sıra

Önce ÖNERİ #10'u uygula (TodoItem ayrı component). Sonra edit özelliğini ekle. Bu sıra mantıklı çünkü her todo'nun kendi isEditing state'i lazım.

📦 Uygulama Sırası — Önerim

Faz 1: Kritik Bug'lar (Mutlaka)

Bu üçünü önce yap. App'in çalışması için gerekli.

- #1 Aynı ID sorununu düzelt
- #2 Form submit preventDefault ekle
- #3 Submit sonrası form reset et

Faz 2: Code Quality

Bu üçü mimari ve kod kalitesini iyileştirir. Production hazır olmak için önemli.

- #4 Routing kaldır
- #5 update() metodunda map kullan
- #6 TodoFilter'ı ikiye böl (TodoFilter + TodoStats)

Faz 3: TodoItem + Inline Edit

Bu kombo en güzelleri. Önce TodoItem'ı ayır, sonra edit ekle.

- #10 TodoItem component'i yarat
- Edit özelliği TodoItem'a ekle (bonus bölüm)

Faz 4: Polish

Bunlar polish — yapmasan da çalışır. Bir sonraki seviye için iyi.

- #7 ReactiveFormsModule kaldır
- #8 crypto.randomUUID() temizle
- #9 onsubmit → onSubmit
- #11 isValid computed + disabled button
- #12 localStorage type guard
- #13 Date object parse
- #14 ToggleTodoInput sadeleştir
- #15 onInputChange validation kaldır

📦 Genel Tavsiyem

Hepsini bir oturuşta yapma. Faz Faz ilerle, her fazın sonunda test et. Önce Faz 1'i bitir, çalıştığından emin ol, sonra Faz 2'ye geç. Bu şekilde bir sorun olursa hangi değişiklikte bozulduğunu daha kolay tespit edersin.

□ Bittiğinde

Tüm fazları bitirdiğinde projeni tekrar gönder. Son review yaparım. Sonra Modül 5 (Signals & Reactivity Derinlemesine)'e geçeriz — `effect()`, `linkedSignal()`, `resource()` gibi modern Angular'ın kalbini işleyeceğiz. □